

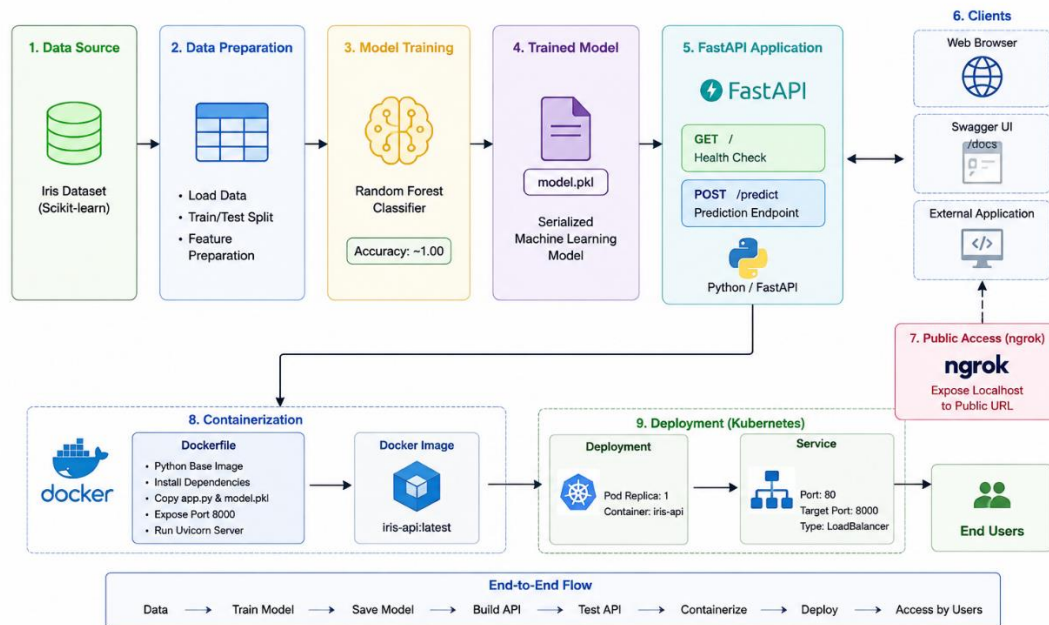
LAB EXPERIMENT - 7

Developing and Deploying APIs for ML Models

Course Outcome: CO2

Architecture

Architecture: Developing and Deploying APIs for ML Models



Step 1 - Install packages

```
!pip install -q fastapi uvicorn scikit-learn pandas joblib pyngrok nest_asyncio requests
```



Step 2 - Train ML model and save it

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import joblib

# Load Iris dataset
iris = load_iris()

X, y = iris.data, iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```

)

# Train model

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_train, y_train)

# Evaluate

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))

# Save model

joblib.dump(model, "model.pkl")

print("model.pkl saved successfully")

```



```

[3] ✓ 6s
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import joblib

# Load Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluate
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))

# Save model
joblib.dump(model, "model.pkl")
print("model.pkl saved successfully")

*** Accuracy: 1.0
model.pkl saved successfully

```

Step 3 – Create FastAPI application

```

from fastapi import FastAPI

from pydantic import BaseModel

import joblib

import numpy as np

# Load trained model

model = joblib.load("model.pkl")

# Create FastAPI app

app = FastAPI(title="Iris Prediction API")

# Input schema

class IrisFeatures(BaseModel):
    sepal_length: float

```

```
    sepal_width: float
    petal_length: float
    petal_width: float
# Home endpoint
@app.get("/")
def home():
    return {"message": "Iris Prediction API is running"}
# Prediction endpoint
@app.post("/predict")
def predict(features: IrisFeatures):
    data = np.array([[
        features.sepal_length,
        features.sepal_width,
        features.petal_length,
        features.petal_width
    ]])
    prediction = model.predict(data)[0]
    species = ["setosa", "versicolor", "virginica"][prediction]
    return {
        "prediction": int(prediction),
        "species": species
    }
```

```

141
✓ 2s
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import numpy as np

# Load trained model
model = joblib.load("model.pkl")

# Create FastAPI app
app = FastAPI(title="Iris Prediction API")

# Input schema
class IrisFeatures(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float

# Home endpoint
@app.get("/")
def home():
    return {"message": "Iris Prediction API is running"}

# Prediction endpoint
@app.post("/predict")
def predict(features: IrisFeatures):
    data = np.array([[
        features.sepal_length,
        features.sepal_width,
        features.petal_length,
        features.petal_width
    ]])

    prediction = model.predict(data)[0]

    species = ["setosa", "versicolor", "virginica"][prediction]

    return {
        "prediction": int(prediction),
        "species": species
    }

```

Step 4: Create a free ngrok account

Open:

- <https://dashboard.ngrok.com/signup>

Sign up using Google/GitHub/email.

Get your auth token

After login, open:

- <https://dashboard.ngrok.com/get-started/your-authtoken>

Copy the token. It looks like: 3HaI2gv34ABIDJnb4h3BQ9XejOK_3xQGpFJrdXLshk6o1tknr

Step 5: Replace PASTE_YOUR_NGROK_TOKEN_HERE with your copied token.

```
from pyngrok import ngrok
```

```
# Add your ngrok auth token here
```

```
ngrok.set_auth_token("3HaI2gv34ABIDJnb4h3BQ9XejOK_3xQGpFJrdXLshk6o1tknr")
```

```
print("ngrok authenticated successfully")
```

```

141
✓ 0s
from pyngrok import ngrok

# Add your ngrok auth token here
ngrok.set_auth_token("3HaI2gv34ABIDJnb4h3BQ9XejOK_3xQGpFJrdXLshk6o1tknr")

print("ngrok authenticated successfully")

*** ngrok authenticated successfully

```

Step 6: Run the server

```
from pyngrok import ngrok

public_url = ngrok.connect(8000)

print("Public URL:", public_url.public_url)
```

```
[7]
✓ 0s
from pyngrok import ngrok

public_url = ngrok.connect(8000)

print("Public URL:", public_url.public_url)

Public URL: https://crinkly-criinge-punctuate.ngrok-free.dev
```

Step 7: Test the API

```
import requests

# Add /predict at the end
url = "https://crinkly-criinge-punctuate.ngrok-free.dev/predict"

sample = {
    "sepal_length": 5.1,
    "sepal_width": 3.5,
    "petal_length": 1.4,
    "petal_width": 0.2
}

response = requests.post(url, json=sample)
print("Status Code:", response.status_code)
print("Response:", response.json())
```

```
[8]
✓ 0s
import requests

# Add /predict at the end
url = "https://crinkly-criinge-punctuate.ngrok-free.dev/predict"

sample = {
    "sepal_length": 5.1,
    "sepal_width": 3.5,
    "petal_length": 1.4,
    "petal_width": 0.2
}

response = requests.post(url, json=sample)

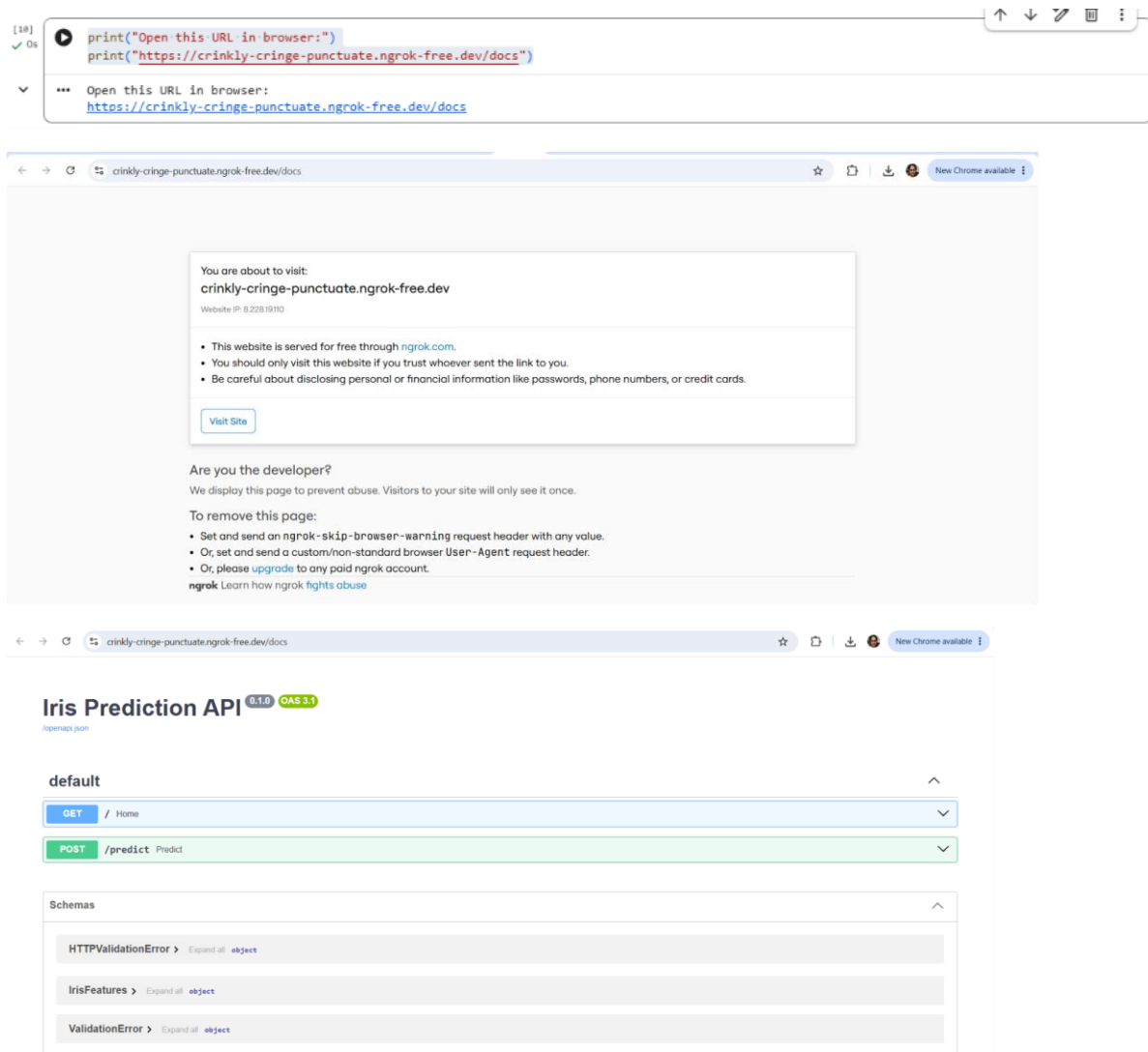
print("Status Code:", response.status_code)
print("Response:", response.json())

INFO:      8.228.19.110:0 - "POST /predict HTTP/1.1" 200 OK
Status Code: 200
Response: {'prediction': 0, 'species': 'setosa'}
```

Step 8: Open Swagger UI

```
print("Open this URL in browser:")

print("https://crinkly-criinge-punctuate.ngrok-free.dev/docs")
```



Step 9: Save the API code as app.py

```
app_code = ""  
  
from fastapi import FastAPI  
from pydantic import BaseModel  
import joblib  
import numpy as np  
model = joblib.load("model.pkl")  
app = FastAPI()  
  
class IrisFeatures(BaseModel):  
    sepal_length: float  
    sepal_width: float  
    petal_length: float
```

```

    petal_width: float
@app.get("/")
def home():
    return {"message": "API Running"}
@app.post("/predict")
def predict(features: IrisFeatures):
    data = np.array([[
        features.sepal_length,
        features.sepal_width,
        features.petal_length,
        features.petal_width
    ]])
    pred = model.predict(data)[0]
    species = ["setosa", "versicolor", "virginica"][pred]
    return {
        "prediction": int(pred),
        "species": species
    }
'''

with open("app.py", "w") as f:
    f.write(app_code)
print("app.py created successfully")

```



```

11 | species = ["setosa", "versicolor", "virginica"][pred]
    |
    | return {
    |     "prediction": int(pred),
    |     "species": species
    | }
    | ...
    |
    | with open("app.py", "w") as f:
    |     f.write(app_code)
    |
    | print("app.py created successfully")
    |
    | ... app.py created successfully

```

Step 10: Create Dockerfile

```

dockerfile = '''
FROM python:3.11-slim
WORKDIR /app

```

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

COPY model.pkl .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

'''

with open("Dockerfile", "w") as f:

 f.write(dockerfile)

requirements = '''

fastapi

uvicorn

scikit-learn

numpy

joblib

pydantic

'''

with open("requirements.txt", "w") as f:

 f.write(requirements)

print("Dockerfile and requirements.txt created")

A screenshot of a code editor window. The editor has a dark theme. On the left, there's a sidebar with a file explorer showing a file named 'Dockerfile'. The main editor area contains Python code. The code starts with a multi-line string for the Dockerfile, followed by a multi-line string for requirements.txt. The code uses 'with open' to write these strings to files. At the end, it prints a message. The code is color-coded: strings are in red, comments in green, and code in blue. The output of the script is visible at the bottom of the editor, showing the message 'Dockerfile and requirements.txt created'.

Step 11: Create Kubernetes deployment file

k8s_yaml = '''

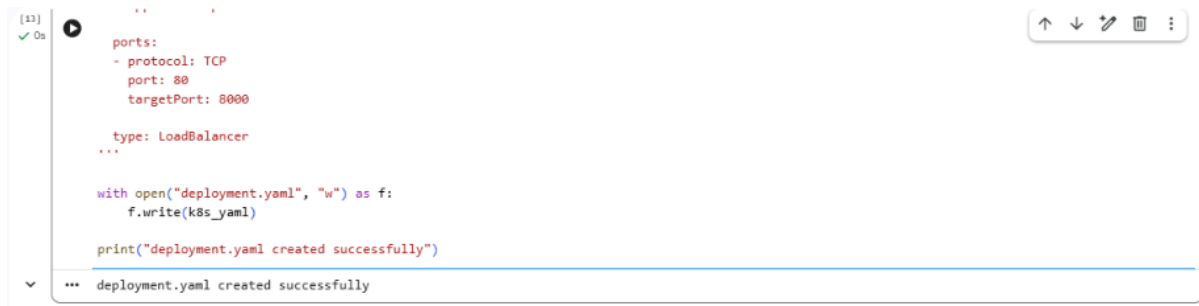
apiVersion: apps/v1

```
kind: Deployment
metadata:
  name: iris-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: iris-api
  template:
    metadata:
      labels:
        app: iris-api
    spec:
      containers:
        - name: iris-api
          image: iris-api:latest
          ports:
            - containerPort: 8000
```

```
apiVersion: v1
kind: Service
metadata:
  name: iris-api-service
spec:
  selector:
    app: iris-api
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8000
  type: LoadBalancer
```

'''

```
with open("deployment.yaml", "w") as f:
    f.write(k8s_yaml)
print("deployment.yaml created successfully")
```



```
[13]
✓ Os
...
ports:
  - protocol: TCP
    port: 80
    targetPort: 8000
...
type: LoadBalancer
...

with open("deployment.yaml", "w") as f:
    f.write(k8s_yaml)

print("deployment.yaml created successfully")

*** deployment.yaml created successfully
```

Step 11: Download all files

```
from google.colab import files
files.download("app.py")
files.download("model.pkl")
files.download("Dockerfile")
files.download("requirements.txt")
files.download("deployment.yaml")
```



```
[14]
✓ Os
from google.colab import files
files.download("app.py")
files.download("model.pkl")
files.download("Dockerfile")
files.download("requirements.txt")
files.download("deployment.yaml")

***
```

You can send requests from Google Colab and then show that ngrok notices them.

Run this cell in Colab:

```
import requests
import time

# Your ngrok API URL
url = "https://crinkly-cringe-punctuate.ngrok-free.dev/predict"

# Sample data
sample = {
    "sepal_length": 6.2,
```

```

    "sepal_width": 3.4,
    "petal_length": 5.4,
    "petal_width": 2.3
}

# Send request
response = requests.post(url, json=sample)

print("Status Code:", response.status_code)
print("Response:", response.json())

```

Wait a moment so ngrok records it

```
time.sleep(2)
```

You should see something like:

Status Code: 200

Response: {'prediction': 2, 'species': 'virginica'}



The screenshot shows a Jupyter Notebook cell with the following Python code:

```

import requests
import time

# Your ngrok API URL
url = "https://crinkly-cringe-punctuate.ngrok-free.dev/predict"

# Sample data
sample = {
    "sepal_length": 6.2,
    "sepal_width": 3.4,
    "petal_length": 5.4,
    "petal_width": 2.3
}

# Send request
response = requests.post(url, json=sample)

print("Status Code:", response.status_code)
print("Response:", response.json())

# Wait a moment so ngrok records it
time.sleep(2)

```

The output of the cell is:

```

INFO:      8.228.19.110:0 - "POST /predict HTTP/1.1" 200 OK
Status Code: 200
Response: {'prediction': 2, 'species': 'virginica'}

```

Now show that ngrok noticed the request

Run this next cell:

```

import json

request_data = latest.get("request", {})

response_data = latest.get("response", {})

```

```
print(json.dumps({
    "method": request_data.get("method"),
    "uri": request_data.get("uri"),
    "remote_addr": request_data.get("remote_addr", "Not available"),
    "status_code": response_data.get("status_code")
}, indent=2))
```

This is excellent evidence that ngrok captured the request.



The screenshot shows a Jupyter Notebook interface. The top part is a code editor with the following Python code:

```
[17]: import json

request_data = latest.get("request", {})
response_data = latest.get("response", {})

print(json.dumps({
    "method": request_data.get("method"),
    "uri": request_data.get("uri"),
    "remote_addr": request_data.get("remote_addr", "Not available"),
    "status_code": response_data.get("status_code")
}, indent=2))
```

Below the code editor, the output is displayed as a JSON object:

```
{
  "method": "POST",
  "uri": "/predict",
  "remote_addr": "Not available",
  "status_code": 200
}
```

Open your ngrok Swagger page:

<https://crinkly-cringe-punctuate.ngrok-free.dev/docs>

Expand:

POST → /predict

Then click **Try it out**.

You should see the input fields corresponding to:

sepal_length

sepal_width

petal_length

petal_width

Enter, for example:

sepal_length = 5.1

sepal_width = 3.5

petal_length = 1.4

petal_width = 0.2

Click **Execute**.

←

→

🔍

crinkly-criinge-punctuate.ngrok-free.dev/docs#/default/predict_predict_post

☆

📄

📌

👤

New Chrome available

GET / Home

POST /predict Predict

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'https://crinkly-criinge-punctuate.ngrok-free.dev/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "sepal_length": 5.1,
    "sepal_width": 3.5,
    "petal_length": 1.4,
    "petal_width": 0.2
  }'
```

Request URL

https://crinkly-criinge-punctuate.ngrok-free.dev/predict

Server response

Code

Details

200

Response body

```
{
  "prediction": 0,
  "species": "setosa"
}
```

Response headers

```
content-length: 35
content-type: application/json
date: Wed, 12 Aug 2025 04:05:41 GMT
ngrok-agent-ip: 8.228.19.118
server: uvicorn
```

Responses

Code

Description

Links

200

Successful Response

No links

Media type

application/json

Controls Accept header

Example Value | Schema

"string"

422

Validation Error

No links

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```